

JAVA

Le stringhe

String

- ▣ Java non possiede un tipo stringa
- ▣ In Java, le stringhe non sono “pezzi di memoria” con dentro dei caratteri, come in C
- ▣ Ogni Stringa è una *istanza di una classe* built-in di Java.
 - ▣ Classe String
 - ▣ Classe StringBuffer
 - ▣ Classe StringTokenizer

La classe String

- ❑ Le stringhe Java **non sono array** di caratteri
- ❑ Sono oggetti concettualmente diversi, cioè non si applicano gli operatori degli array!
 - ❑ in particolare, non si applica il classico []
- ❑ Le stringhe sono sequenze di caratteri **immutabili**
- ❑ Dopo che una stringa è stata costruita il suo contenuto non può essere modificato
- ❑ Fornisce numerosi metodi per lavorare con le stringhe
 - ❑ Operazioni di base
 - ❑ Confronti fra stringhe
 - ❑ Costruzioni di stringhe correlate
 - ❑ Conversione di stringhe

COSTANTI `String`

- Le costanti `String` possono essere denotate nel modo usuale: `"ciao"` `"mondo\n"`
- Quando si scrive una costante `String` tra virgolette, viene creato implicitamente un nuovo oggetto di classe `String`, inizializzato a tale valore.
- Una costante `String` non può eccedere la riga: quindi, dovendo scrivere stringhe più lunghe, conviene spezzarle e concatenarle con `+`.

String

- Java fornisce alcuni supporti *extra per gli oggetti della classe String*
 - per ragioni di convenienza, dato che le stringhe sono frequentemente utilizzate, esse appaiono quasi come dei tipi primitivi ma non lo sono
- Tre caratteristiche di String:
 - Non è necessario una istanziazione esplicita con la parola chiave **new**
 - String supporta l'operatore overloaded '+' per supportare la concatenazione
 - esistono numerosi metodi predefiniti nella class built-in *String*
- Una stringa Java rappresenta uno specifico valore e come tale è imm modificabile: una **String** non è un contenitore!
 - se occorre un contenitore esiste **StringBuffer**

Costruttori

- `public String()`
- `public String (String value)`
- `Public String (StringBuilder value)`

Esempi di creazione di stringhe:

```
String newString = new String(stringLiteral);
```

```
String message = new String("Welcome to Java!");
```

```
String message = "Welcome to Java!";
```

```
String message = message + "prova"
```

Stringhe in Java

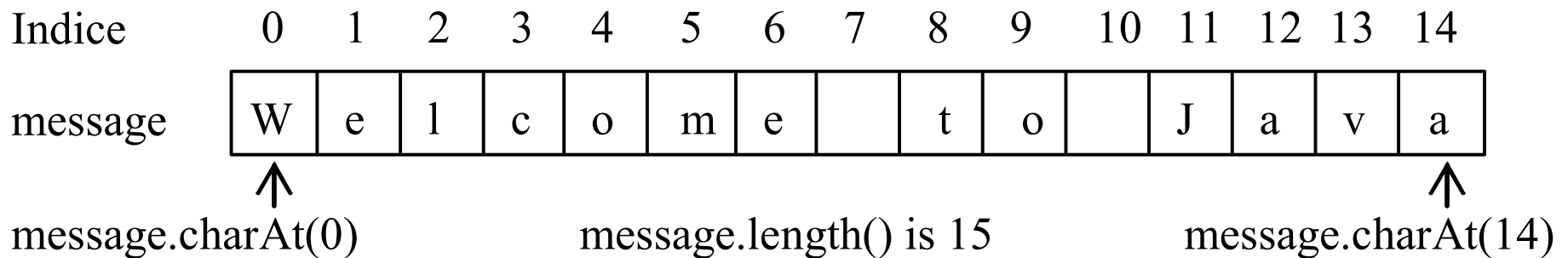
- Per selezionare il carattere i-esimo si usa il metodo `charAt()`:

```
String message = "Welcome to java";  
char ch = message.charAt(4);
```

❑ `message[0]` errore!!!

❑ `message.charAt(index)`

❑ L'indice inizia da 0



Lunghezza di una stringa

La lunghezza di una stringa può essere ricavata usando il metodo
`length()`

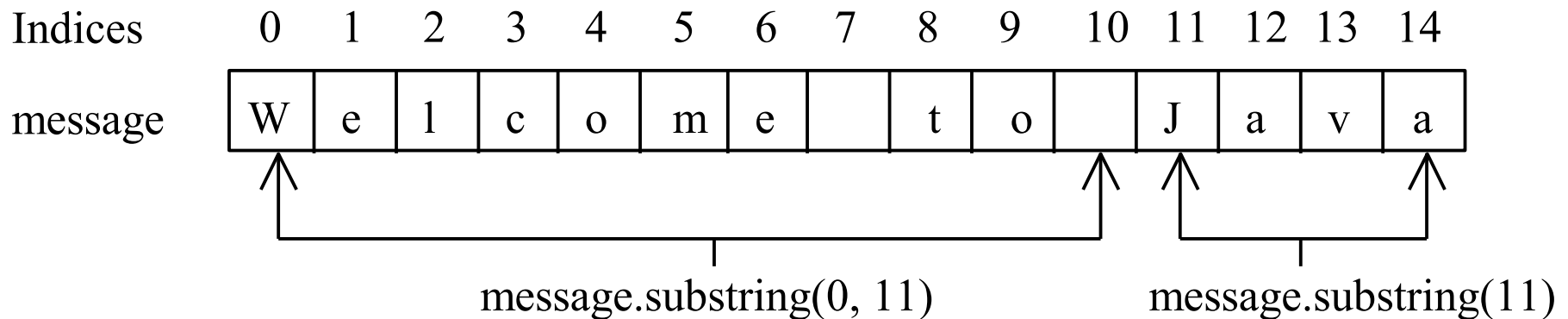
```
message = "Welcome";
```

```
message.length() (ritorna il valore 7)
```

Estrazione di Substrings

```
String s1 = "Welcome to Java";
```

```
String s2 = s1.substring(0, 11) + "HTML";
```



Il secondo argomento e' il primo indice non incluso nella sottostringa

Concatenazione

```
String s3 = s1.concat(s2);
```

```
String s3 = s1 + s2;
```

```
System.out.println("tre piu cinque "+ 3 + 5);
```

output:

tre piu cinque 35

```
System.out.println("tre piu cinque " + (3 + 5));
```

output:

tre piu cinque 8

```
System.out.println(3 + 5);
```

output:

8

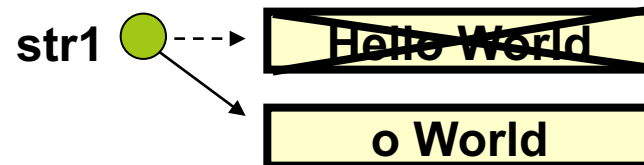
Modificabilità di una stringa

- Non si può modificare il contenuto di un oggetto stringa (si dice che una stringa è immutabile)
- Cosa accade in memoria se si vuole modificare una stringa
 1. viene creato nuovo oggetto String
 2. il riferimento viene aggiornato
 3. la vecchia stringa viene eliminata dal 'garbage collector'
- Per esempio:

String str1 = "Hello World"



str1 = str1.substring(4)



La classe *StringBuffer* supera questa limitazione

Confronto fra stringhe

String method `equals()`

String method `equalsIgnoreCase()`

Output:
false
true

```
String str1 = "HeLlO";  
String str2 = "hello";  
System.out.println(str1.equals(str2));  
System.out.println(str1.equalsIgnoreCase(str2));
```

Confronto fra stringhe

String method: **compareTo(String)**

a.compareTo(b) ritorna neg if $a < b$
 ritorna 0 if a equals b
 ritorna pos if $a > b$

Uso:

```
String str1, str2;  
...  
if (str1.compareTo(str2) < 0) {  
    // str1 is alphabetically 1st  
} else if (str1.compareTo(str2)==0) {  
    // str1 equals str2  
} else { // implies str1 > str2  
  
// str1 is alphabetically 2nd  
}
```

Note

- Definizione di una classe di esempio Box

```
public class Box{
    private int length;
    private int width;
    private int height;

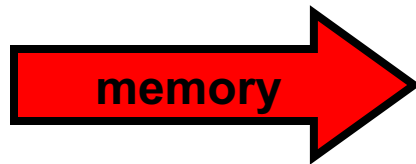
    public Box(int l, int w, int h){
        length = l;
        width = w;
        height = h;
    }

    public boolean equals(Box b){
        if(length == b.getLength() && width == b.getWidth() && height ==
b.getHeight())
            return true;
        else
            return false;
    }
}
```

Oggetti e riferimenti: il caso standard

```
box1 = new Box(1, 2, 3);  
box2 = new Box(8, 5, 7);  
box1 = box2;  
System.out.println(box1 == box2);           // stampa true
```

```
System.out.println(box1.equals(box2));       // stampa true
```



box1



box2



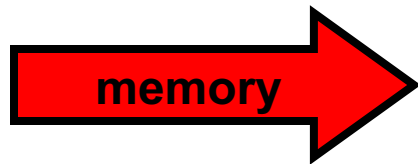
Oggetti e riferimenti: il caso standard

```
box1 = new Box(1, 2, 3);
```

```
box2 = new Box(1, 2, 3);
```

```
System.out.println(box1 == box2);           // stampa false
```

```
System.out.println(box1.equals(box2));       // stampa true
```



box1



L=1, W=2, H=3

box2



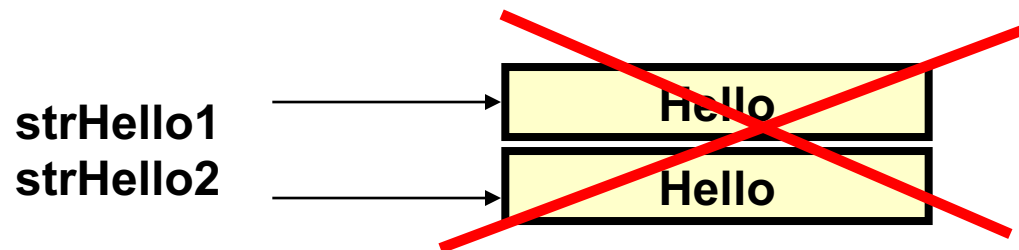
L=1, W=2, H=3

Uguaglianza con Riferimenti e Oggetti: String è speciale?

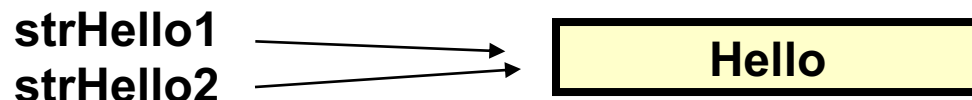
Strings ha una caratteristica speciale. In alcuni casi, ma non in tutti, è possibile utilizzare `'=='` *per confrontare due Strings, oltre che equals()*. *Esempio:*

```
String strHello1 = "Hello";  
String strHello2 = "Hello";
```

Cio' che accade in memoria non e'



ma



Uguaglianza con Riferimenti e Oggetti: Strings e' speciale

```
String strHello1 = "Hello";  
String strHello2 = "Hello";
```

Il compilatore è “furbo” abbastanza per riconoscere che le due stringhe sono identiche. Quindi decide di risparmiare memoria ed utilizzare la stessa locazione di memoria. I due riferimenti **strHello1 e **strHello2** puntano alla stessa locazione di memoria.**

Lo stesso risultato si ottiene se noi scriviamo:

```
String strHello2 = "Hell" + "o";
```

Cio significa che per il seguente codice equals() e '==' operano allo stesso modo:

[illegible]

Eccezione all'eccezione di String

- ... ma questo caso speciale per '==' nel confronto fra oggetti String
NON SEMPRE FUNZIONA . . .
- Se un oggetto String è creato con l'uso della parola chiave 'new', i due oggetti String non occupano lo stesso spazio di memoria.
- Quindi '==' può restituire false, ma equals() può restituire true, se i contenuti delle stringhe sono gli stessi.
- Pertanto c'è una **eccezione nell'eccezione**.
- Come conviene risolvere il problema di sapere quando "==" si comporta come equals()?
- **Non usare l'eccezione**. Non confrontare String con '=='
- Utilizzare '==' per confrontare solo tipi primitivi
- Utilizzare equals per confrontare oggetti.

Metodo per la gestione minuscolo-maiuscolo

- ❑ **`equalsIgnoreCase()`**
esegue il test di uguaglianza fra due oggetti `String` ignorando il case
- ❑ **`toUpperCase()`**
crea un versione della stringa con caratteri maiuscoli (uppercase)
- ❑ **`toLowerCase()`**
crea un versione della stringa con caratteri minuscoli (lowercase)
- ❑ **Note: nessuno di questi modifica la stringa originale.**

toUpperCase() e toLowerCase()

```
String str = "John Lennon";  
String strSmall = str.toLowerCase();  
System.out.println(str);  
System.out.println(strSmall);  
System.out.println(str.toUpperCase());  
System.out.println(str);
```

Output:
John Lennon
john lennon
JOHN LENNON
John Lennon

Ricerca di un pattern: indexOf(String)

```
String str = "catfood";  
int location = str.indexOf("food");  
System.out.println("pattern food begins at " +  
    location);  
System.out.println("pattern dog begins at " +  
    str.indexOf("dog"));
```

catfood
0123456

Output:

pattern food inizia in 3
pattern dog inizia in -1

-1 è ritornato quando
viene trovato il pattern !

Ricerca di un pattern: indexOf(String, int)

Può specificare l'indice di partenza della ricerca per trovare tutte le occorrenze di un pattern!

```
String str = "abracadabra abracadabra";  
int index = str.indexOf("abra");  
while (index != -1) {  
    System.out.println("found at " + index);  
    index = str.indexOf("abra", index + 1);  
} // il -1 finale non viene stampato
```

Output:
found at 0
found at 7
found at 12
found at 19

Metodi per String

- Tutte le classi Java definiscono un metodo `toString()` che produce una `String` a partire da un oggetto della classe: ciò consente di “stampare” facilmente qualunque oggetto di qualunque classe
- È responsabilità del progettista definire un metodo `toString()` che produca una stringa “significativa”
- Quello predefinito stampa un identificativo alfanumerico dell’oggetto.

ESEMPIO

- ▣ Converte ch in stringa e lo concatena alla frase.
- ▣ Usa il metodo toString() predefinito di Counter → Stampa un identificativo dell'oggetto c.

```
public class Esempio5 {  
    public static void main(String args[]){  
        String s = "Nel mezzo del cammin";  
        char ch = s.charAt(4);  
        System.out.println(ch);  
        System.out.println("Carattere: " + ch);  
        Counter c = new Counter(10);  
        System.out.println(c);  
    }  
}
```

VARIANTE

- La stampa di poco fa non vi piace?
- Potete ridefinire esplicitamente il metodo `toString()` della classe `Counter`, facendogli stampare ciò che preferite.

Ad esempio:

```
public class Counter {  
    ...  
    public String toString(){  
        ritorna "Counter di valore " + val;  
    }  
}
```

VARIANTE

- ▣ Lo stesso identico esempio:

```
public class Esempio5 {  
    public static void main(String args[]){  
        String s = "Nel mezzo del cammin";  
        char ch = s.charAt(4);  
        System.out.println(ch);  
        System.out.println("Carattere: " + ch);  
        Counter c = new Counter(10);  
        System.out.println(c);  
    }  
}
```

La classe StringBuffer

StringBuffer

- `StringBuffer` è una alternativa a `String`
- `StringBuffer` può essere utilizzata dove è usata `string`.
- `StringBuffer` è più flessibile di `String`.
 - `add`, `insert`, o `append` nuovi contenuti

StringBuffer

- `StringBuffer append(char[] data)`
- `StringBuffer append(char[] data, int offset, int len)`
- `StringBuffer append(aPrimitiveType v)`
- `StringBuffer append(String str)`
- `int capacity()`
- `char charAt(int index)`
- `StringBuffer delete(int startIndex, int endIndex)`
- `StringBuffer deleteCharAt(int index)`
- `StringBuffer insert(int inde, char[] data, int offset, int len)`

StringBuffer

- `StringBuffer insert(int offset, char[] data)`
- `StringBuffer insert(int offset, aPrimitiveType b)`
- `StringBuffer insert(offset: int, str: String)`
- `int length()`
- `StringBuffer replace(int startIndex, int endIndex, String str)`
- `StringBuffer reverse(): StringBuffer`
- `void setCharAt(int index, char ch)`
- `void setLength(int newLength)`
- `StringBuffer substring(int start)`
- `StringBuffer substring(int start, int end)`

Costruttori di StringBuffer

- `public StringBuffer()`
 - Capacità iniziale 16 caratteri.
- `public StringBuffer(int length)`
 - Capacità iniziale uguale ad un numero di caratteri `length`.
- `public StringBuffer(String str)`
 - Rappresenta la stessa sequenza di caratteri di `str`.
capacità iniziale uguale ad un numero di caratteri pari a 16 più la lunghezza di `str`.

Append nuovi contenuti ad una String Buffer

```
StringBuffer strBuf = new StringBuffer();  
strBuf.append("Welcome");  
strBuf.append(' ');  
strBuf.append("to");  
strBuf.append(' ');  
strBuf.append("Java");
```

La classe StringTokenizer

Costruttori di StringTokenizer

- `StringTokenizer(String s, String delim, boolean returnTokens)`
- `StringTokenizer(String s, String delim)`
- `StringTokenizer(String s)`

Metodi di StringTokenizer

- `boolean hasMoreTokens()`
- `String nextToken()`
- `String nextToken(String delim)`
- `int countTokens()`
- `Boolean hasMoreTokens()`
- `String nextToken()`
- `String nextToken(String delim)`